Dominic Lim *Soma Capital case study*

A written submission with a runnable prototype of the trust core behind it (about 2,000 lines of TypeScript, no runtime dependencies, 39 tests). The dashboard runs it live, and technical claims cite a `file:line`. You should be able to read this straight through without opening any of it.

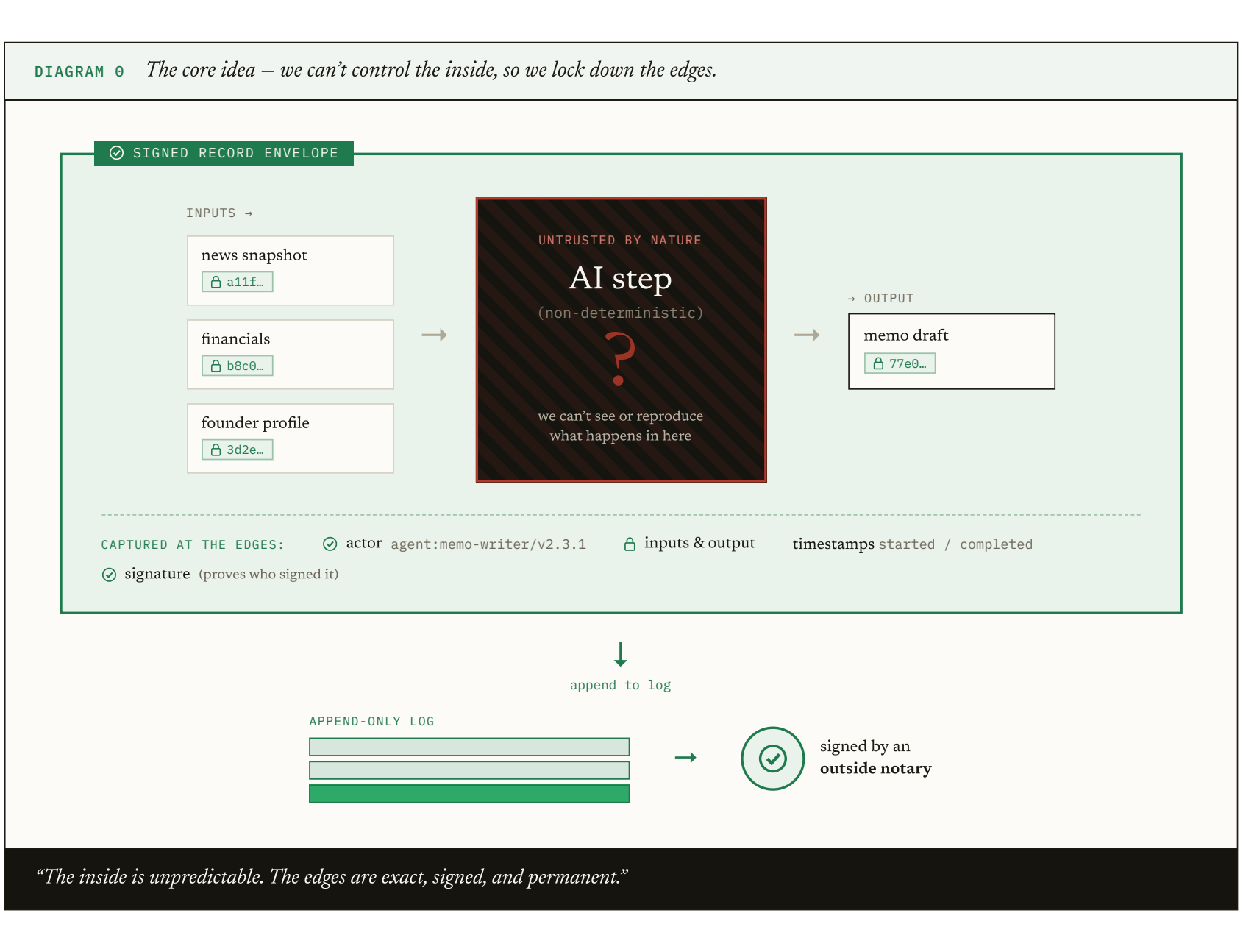
Approach

The design turns on one decision: you can't make an LLM deterministic, so don't try. Make the *record* of what it did deterministic instead. Let the model be as unpredictable as it wants inside a step. The moment anything crosses the step's edge (the inputs it read, the output it produced, who ran it, when) capture it exactly, sign it, and append it to a log that can't be rewritten. The unpredictable part stays sealed inside the step; everything observable from outside is exact, attributed, and permanent.

"Can't be rewritten" is the load-bearing phrase. An ordinary database is append-only by *policy*, so anyone with admin access can edit a row and cover their tracks. This design is append-only by *math*: records are chained by hash so that changing any past record changes one fingerprint at the top, and that top fingerprint is signed by an outside party on a schedule. That turns "trust us, we didn't touch it" into "here's the proof, check it yourself."

Everything below follows from that one move, so it's worth being explicit about the kind of problem it leaves. At the brief's two-year projection of 50,000 actions a day, the system takes under one write per second, so raw throughput is never the constraint. The hard part is trust, and specifically distributed-systems trust: the guarantee has to survive a compromised server, a dishonest insider, and an auditor who has no reason to believe anything Soma says. The whole architecture is organized around surviving those three.

DIAGRAM 0 *The core idea — we can't control the inside, so we lock down the edges.*



"The inside is unpredictable. The edges are exact, signed, and permanent."

---

1 High-Level Architecture

The components fall out of the requirements

I didn't start by choosing components. I started by asking what the system has to be able to prove, and the pieces followed from that.

To answer *has this been modified*, an artifact can't be identified by a name that a person can later point at different bytes. It has to be identified by its bytes. That forces the first component, a **content-addressed store**, where an artifact's address is a fingerprint of its content, so a modified artifact is simply a different artifact with a different address.

To answer *who made this, from what, and when*, every step needs a small **record** holding its actor, its inputs, and its output, signed so the claim is attributable.

To make *has this been modified* hold for the records too, not only the artifacts, those records can't sit in a plain database where an administrator can edit a row. They need to live in something append-only by construction and checkable from the outside, which is the **transparency log**. And because even the log's own operator could rewrite it, an **external anchor** periodically pins the log's state somewhere Soma doesn't control.

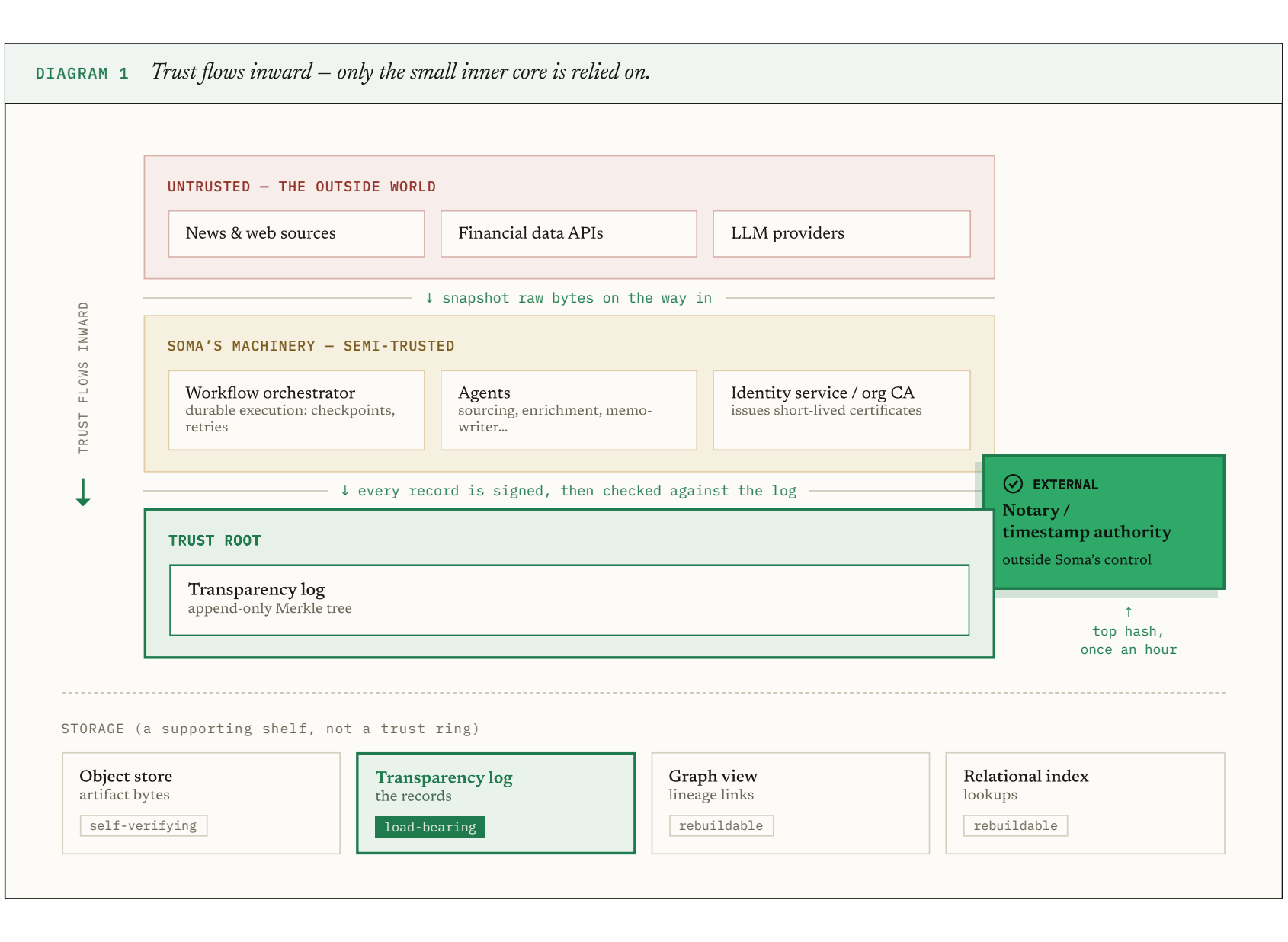
For records to be signed, actors need cryptographic identities and keys, which is the **identity service**. Agents also fail, time out, and retry across runs that can last days, and none of that can be allowed to corrupt the record, which is the job of a **durable workflow orchestrator**. Finally, answering an auditor's question means walking the records back from a decision and handing over something checkable, which is the **audit layer**.

Six components, each one forced by a requirement rather than chosen for its own sake. The rest of this section is how they relate.

Trust boundaries

The most important question in a design like this is what you are allowed to trust, because a guarantee is only ever as strong as the weakest thing it depends on. The answer here is drawn as three rings.

DIAGRAM 1 *Trust flows inward — only the small inner core is relied on.*


UNTRUSTED — THE OUTSIDE WORLD
News & web sources
Financial data APIs
LLM providers
↓ snapshot raw bytes on the way in
SOMA'S MACHINERY — SEMI-TRUSTED
Workflow orchestrator
durable execution: checkpoints, retries
Agents
sourcing, enrichment, memo-writer…
Identity service / org CA
issues short-lived certificates
↓ every record is signed, then checked against the log
TRUST ROOT
Transparency log
append-only Merkle tree
TRUST FLOWS INWARD
EXTERNAL
Notary / timestamp authority
outside Soma's control
top hash, once an hour
STORAGE (a supporting shelf, not a trust ring)
Object store
artifact bytes
self-verifying
Transparency log
the records
load-bearing
Graph view
lineage links
rebuildable
Relational index
lookups
rebuildable

The outer ring is everything Soma doesn't control: news sites, financial data APIs, the LLM providers. It can be wrong, slow, or hostile, so nothing it says is taken as true. We snapshot exactly what it returns and treat that snapshot as a faithful record of what was received, never as a fact about the world.

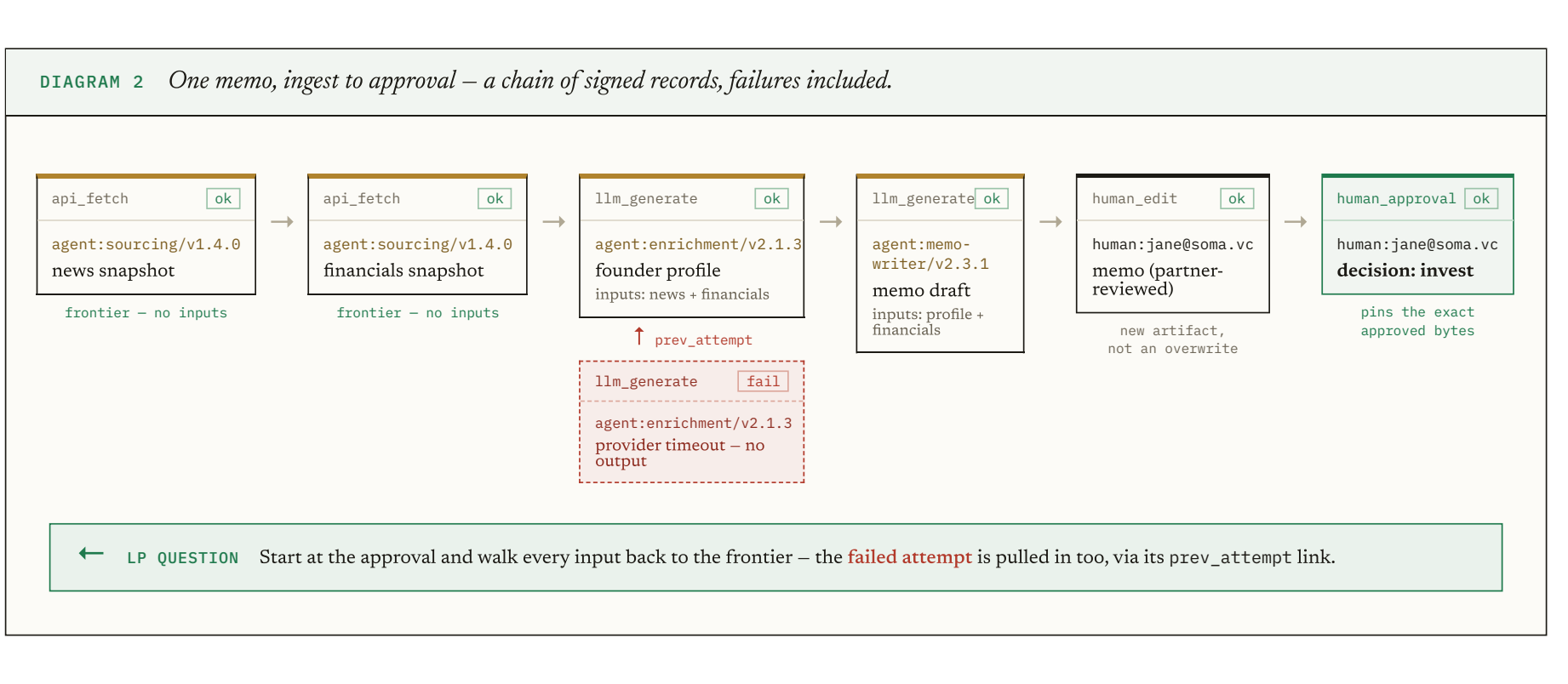
The middle ring is Soma's own machinery: the orchestrator, the agents, the identity service. It does the real work and it holds signing keys, and the entire point of the design is to avoid having to trust it. We assume any single machine in it could be compromised, and we require that a lie told inside the middle ring is caught by the inner ring.

The inner ring is the trust root: the append-only log and the external notary. It's kept deliberately small, because it's the one thing everyone ultimately relies on, and a small trust root is one you can actually reason about end to end. Trust flows inward. The outer ring proves nothing, the middle ring's output is signed and then checked, and the inner ring is held honest by math and by an outsider.

Following one memo through the system

The clearest way to see the components work together is to follow a single memo from raw sources to an approved investment decision, and then watch it get audited. This is the exact path the prototype runs.

DIAGRAM 2 *One memo, ingest to approval — a chain of signed records, failures included.*



← LP QUESTION Start at the approval and walk every input back to the frontier — the failed attempt is pulled in too, via its `prev_attempt` link.

A sourcing agent fetches a news article and a financials response and saves the raw bytes of each. Those two saves are the first records, and they have no inputs, which makes them the frontier: the edge of the system where outside data entered and everything downstream traces back to. An enrichment agent reads both snapshots and writes a founder profile. Its first attempt times out, and that failure is written down as its own record; the retry that succeeds points back at it, so the history keeps the failure instead of quietly erasing it. A memo agent drafts from the profile. A partner edits the draft, which creates a new document rather than overwriting the old one, because an overwrite would destroy the very history we're trying to keep. The partner approves, and the approval record pins the exact bytes approved. Later the LP asks the brief's headline question, and the system starts at that approval and walks the input links all the way back to the two frontier snapshots.

That walk is the whole system in miniature: the artifacts came from the content store, each step is a signed record, the records are ordered in the log, and the audit layer reconstructs the chain on demand.

Durable workflows, and the boundary the brief asks about

The brief singles out the boundary between deterministic and non-deterministic steps, and it's worth being precise about why that boundary is the genuinely hard part.

Reliable execution on its own is a solved, purchasable problem. A durable workflow engine (Temporal and its relatives) checkpoints a workflow's progress so a crash resumes where it left off, and uses idempotency keys so a retried database write happens once instead of twice. Rebuilding that would be reinventing mature infrastructure, so I don't. What I own is the one place where reliability and provenance collide, which is retries.

A retry means a step runs more than once, and the two kinds of step have to be handled as opposites. A **deterministic** step, like a database write or an API call, can be re-run safely, because doing it again produces the same effect and the idempotency key absorbs the duplicate. A **non-deterministic** step, like an LLM call, can't be re-run to check it, because running it again produces a different answer. So on a retry you never re-execute a non-deterministic step to verify it; you record its output once and reuse that record.

The consequence for provenance is the decision that matters. Every attempt, including the failed one, becomes its own permanent record, and the successful retry links back to the failure. Reliability is handled by resuming; provenance is preserved by never letting a retry hide what already happened. As the deep dive shows, a history that drops that failed attempt fails verification, so the boundary is enforced by the math, not just described in a doc.

Identity, and the trust model

A signature only means something if you know whose key produced it and that the key couldn't have been stolen and quietly used for months. That's an identity problem, and the obvious approach to it is the wrong one.

The obvious approach is to give each agent a long-lived key and keep a revocation list for when a key leaks. I rejected it, because revocation is the part of every such system that fails in practice: a stolen key stays valid until a human notices and acts, and that window is exactly when the damage is done. So instead of long-lived keys, an organization CA issues **short-lived certificates**, good for hours, to each agent instance. Rotation then takes care of itself, because a running agent simply requests a fresh certificate as its current one nears expiry, and there's no long-lived secret lying around to steal. If a key is known to be compromised inside its short window, every issued certificate is itself logged, so it can still be denylisted for emergencies.

That trade has a cost, and naming it is the honest part: it needs an always-on issuing service, and if that service is down, agents can't sign. I took it because, at the rate agents work, keeping an issuer available is a far easier operational problem than keeping a revocation list both correct and fast. For an identity that had to sign while offline, the trade would flip and long-lived keys would win.

Two more pieces finish the model. An agent never acts on its own authority: its certificate must carry a delegation chain that terminates at a real human, or verification rejects it, so an accountable person always sits at the root of any automated chain. And holding a valid key is not enough to sign as someone else, a property the deep dive returns to, because the identity claimed in a record has to match the certificate the CA actually issued. Humans sit outside the agent machinery entirely, authenticating through Soma's existing SSO with hardware-backed keys.

The trust model reduces to a sentence: trust the org CA for who is who, trust an outside notary for when history existed, and trust nothing else, including Soma's own servers.

Storage, and what actually has to be trusted

TABLE 2 *Four storage layers — only one is load-bearing.*

LAYER	WHAT IT HOLDS	DO YOU HAVE TO TRUST IT?
Object store	artifact bytes	**No** — self-verifying (the address is the hash)
Transparency log	the records	Yes — this + the outside anchor is the only load-bearing layer
Graph view	the lineage links	**No** — rebuildable from the log
Relational index	lookups	**No** — rebuildable from the log

There are four storage layers, and the reason to separate them is that only one of them has to be trusted. Artifact bytes sit in ordinary object storage, self-verifying because the address is the hash. The records sit in the append-only log, which together with the external anchor is the only load-bearing layer. A graph view makes the audit walk fast, and a relational index answers ordinary "everything by this agent this month" lookups, and both are derived: if either were corrupted or distrusted, you'd rebuild it by replaying the log. That's the property worth designing for. An auditor who trusted none of Soma's databases could throw the derived layers away, rebuild them from the log, and every proof would still hold.

The five questions, answered plainly

The brief's first requirement is really five questions each artifact must answer, so here they are in plain language, with the machinery that makes each true deferred to the deep dive.

TABLE 1 *The five provenance questions, answered in their words.*

"Which agent or human created this?"	Every step names its actor — a person or a specific agent version — signed in, so nobody can put someone else's name on their work.
"What inputs or sources were used?"	The record lists every input, each pointing at the exact earlier artifact. Follow the pointers back for the full family tree.
"What version of the agent produced it?"	The version is in the actor's name; for AI steps the record also stores the model, prompt, and settings.
"When was it created?"	Start and completion times are in the record — claimed by the actor, with the notary proving when a whole batch of history existed.
"Has it been modified?"	An artifact's name *is* the fingerprint of its bytes — change it and the name changes. Re-checked on every read.

---

2 Deep Dive: the provenance record, the DAG, and the log

The brief asks for depth on one component. I chose this core, because it's where "who made this, from what, and has it changed" is actually answered, and because it's the part where a plausible-looking design can be quietly, dangerously wrong. Three things do the work together: the record that captures one step, the way records link into a graph that gives lineage, and the log that makes the whole history complete and unrewritable. I'll build each up from the problem it solves.

What a record has to capture, and why

Rather than open with a schema, start from the five questions and let the record's contents be forced by them, because that's the only way to be sure the record is neither missing something nor carrying dead weight.

Who made this forces an actor identity and a signature over the whole record, so the claim is both attributable and unforgeable. *From what inputs* forces a list of the exact upstream artifacts the step read, each referenced by its content fingerprint. *Which version* forces the agent's version into its name, and for an AI step, the model and its settings. *When* forces a start and completion time. And *has it been modified* comes for free from referencing the output by fingerprint, because any change to the output changes its fingerprint and breaks every record that pointed at it.

Two choices inside the record earn their keep. It references inputs and outputs by fingerprint rather than embedding the bytes, so a record stays about a kilobyte whether the artifact is a 100-byte CRM update or a 50 KB memo, and so that deleting an artifact's bytes for a privacy request never disturbs the records that reference it. And a failed attempt is a first-class record that the successful retry links back to, which is what makes hiding a failure structurally impossible rather than merely discouraged.

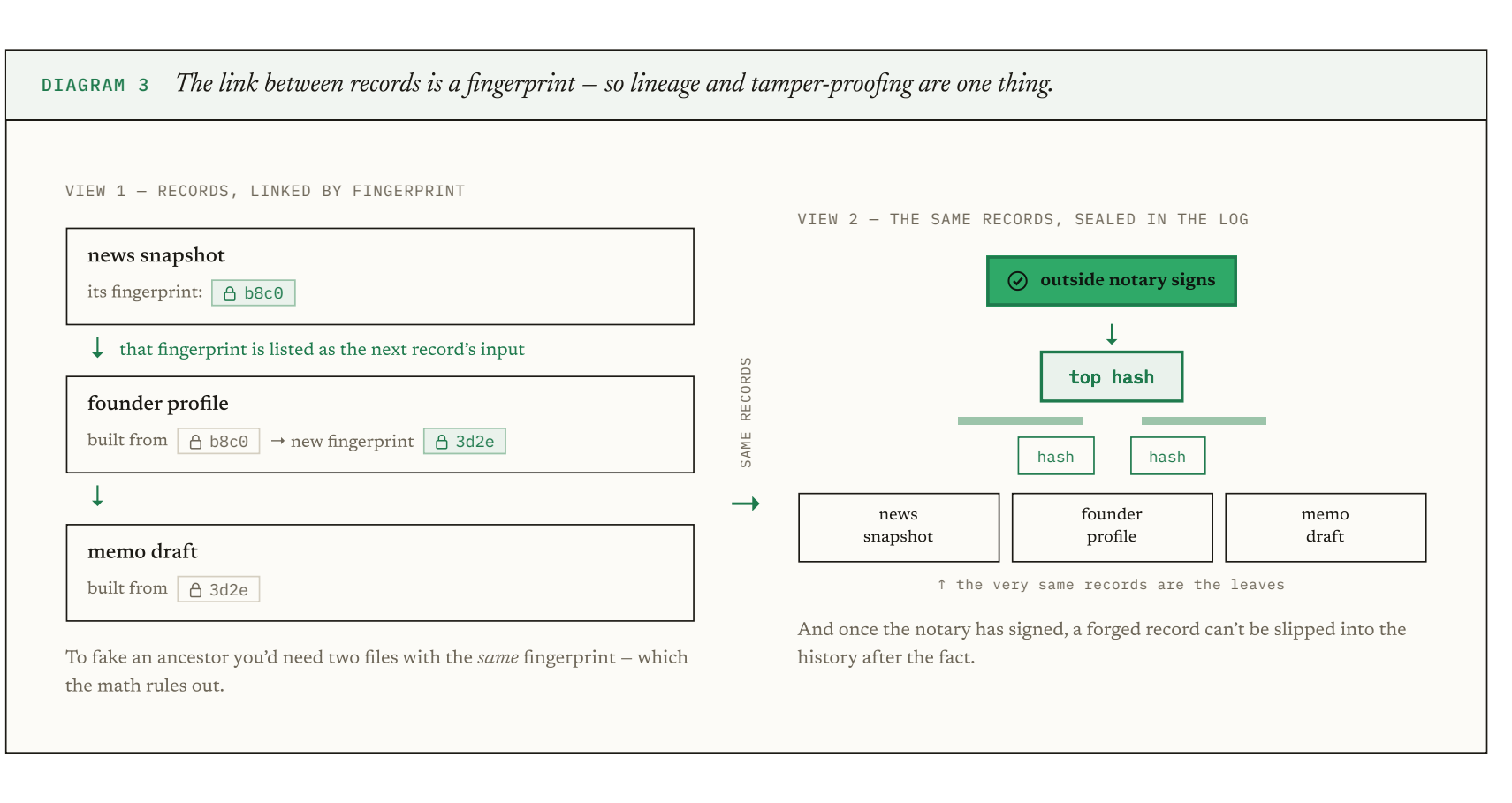
One field exists specifically for the AI problem, and it's worth pausing on, because it's the concrete form of "wrapping a non-deterministic step in a deterministic guarantee." We can't record *why* a model produced a given output; that lives inside a black box we don't control. What we can record is everything that went *into* the call: the model and version, the exact prompt, the sampling settings, and the provider's own request ID. So the step becomes fully attributable even though it can never be reproduced. That distinction, attributable but not reproducible, is the honest center of putting AI into an audit trail, and it's why the design never pretends it can re-run a model to check it (the tradeoffs return to this).

Hash-links are the lineage

This is where individual records become a history. Each record names its inputs by fingerprint, and each of those inputs was the output of an earlier record that named *its* inputs by fingerprint, back to the frontier. So the records form a graph, and the edges of that graph are fingerprints sealed inside signed records.

That detail is the entire reason to build it this way. The ordinary way to store lineage is a table with rows saying "memo X came from draft Y," which anyone with database access can quietly edit. Here, faking an ancestor would need either two different documents with the same fingerprint, which the hash function is designed to make infeasible, or a newly forged signed record, which can't be slipped into the log after the fact. Lineage and tamper-resistance stop being two systems you have to keep honest with each other. They become the same mechanism.

DIAGRAM 3 *The link between records is a fingerprint — so lineage and tamper-proofing are one thing.*


VIEW 1 — RECORDS, LINKED BY FINGERPRINT
news snapshot
its fingerprint: b8c0
that fingerprint is listed as the next record's input
founder profile
built from b8c0 → new fingerprint 3d2e
memo draft
built from 3d2e
SAME RECORDS
VIEW 2 — THE SAME RECORDS, SEALED IN THE LOG
outside notary signs
top hash
hash
hash
news snapshot
founder profile
memo draft
↑ the very same records are the leaves

To fake an ancestor you'd need two files with the *same* fingerprint — which the math rules out.

And once the notary has signed, a forged record can't be slipped into the history after the fact.

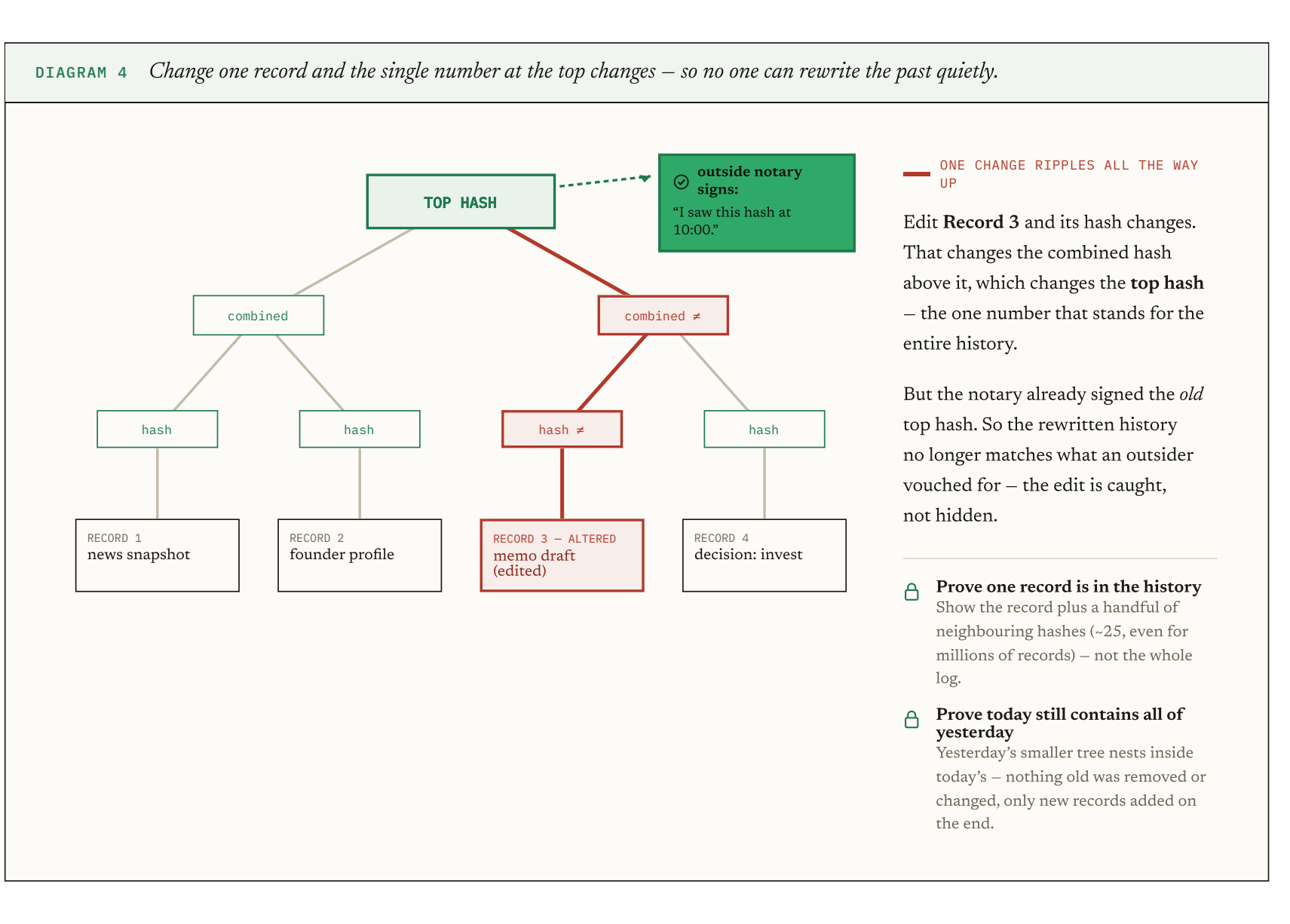
Making the history complete and unrewritable

The graph proves each record is genuine and shows what it came from. It does not prove the history is *complete*. A signature stops a record from being changed, but it does nothing to stop a record from being deleted, and nothing to stop someone from keeping a second, fully-signed, entirely invented history off to the side. You could run two sets of books.

So the real question is how to store the growing pile of records so that no one, including whoever runs the storage, can delete one or swap in a fake history without it being obvious. Publishing a running list doesn't do it, because whoever controls the list can rewrite and republish it, and publishing means nothing unless someone pinned down what it said before.

The mechanism that does work is worth building one step at a time. Hash every record. Pair up those hashes and hash each pair together. Pair up the results and hash again. Keep going until a single hash is left at the top. That top hash is a fingerprint of the entire history at once: change any record anywhere below it, and the top hash changes. This structure is a Merkle tree, and it's the same construction that keeps the world's website certificates honest under Certificate Transparency, which is good evidence it holds up against real adversaries at scale.

DIAGRAM 4 *Change one record and the single number at the top changes — so no one can rewrite the past quietly.*



Edit **Record 3** and its hash changes. That changes the combined hash above it, which changes the **top hash** — the one number that stands for the entire history.

But the notary already signed the *old* top hash. So the rewritten history no longer matches what an outsider vouched for — the edit is caught, not hidden.

Prove one record is in the history Show the record plus a handful of neighbouring hashes (~25, even for millions of records) — not the whole log.

Prove today still contains all of yesterday Yesterday's smaller tree nests inside today's — nothing old was removed or changed, only new records added on the end.

Two capabilities fall out of the tree almost for free, and they happen to be exactly the two an auditor needs. You can prove one record is in the history without handing over the whole thing: give the record plus the handful of sibling hashes along its path to the top, about 25 of them even for millions of records, and the recipient recomputes the top hash and checks it. And you can prove today's history still contains all of yesterday's, unchanged: given yesterday's top hash, there's a short proof that today's tree holds everything from yesterday with new records only appended on the end, and if anything old had been deleted or altered, no such proof could exist.

One hole is left, and closing it is why the design reaches outside Soma at all. All of that assumes someone actually kept yesterday's top hash. If Soma controls everything, it could rewrite the past and simply lie about what the old top hash was. So the top hash can't live only inside Soma. Once an hour, the current top hash is handed to an outside timestamping notary whose one job is to sign "I saw this hash at this time," the same neutral third party used to timestamp legal documents. From that moment the top hash is pinned somewhere Soma can't reach, and any later attempt to rewrite even one old record fails to produce a valid consistency proof back to the hash the notary signed. Rewriting history stops being something forbidden by policy and becomes something detectable by arithmetic.

Answering the LP with a proof that needs no trust in Soma

Now the brief's headline question resolves cleanly, and the shape of the answer is the point. The audit layer takes the approval artifact, walks back through the input links to the frontier, pulling failed attempts along by their links, and packages the result as a proof bundle: every record in the chain with its inclusion proof, the current and last-anchored top hashes with a consistency proof between them, every certificate involved, the notary's receipt, and optionally the document bytes.

What makes that a proof rather than a report is what an outsider can do with it. A standalone verifier checks the entire bundle while trusting exactly two public keys, the org CA and the notary, and sharing no database or code path with the system that produced the bundle. It confirms the top-hash signatures, the external anchor, the append-only consistency since that anchor, each record's presence in the log and its signature and its identity binding, each certificate's chain to the CA, that the lineage closes with no dangling inputs, and that any disclosed bytes still match their fingerprints. If the bundle is malformed in any way it returns a failure instead of crashing, so a crash can never be mistaken for a pass.

The four ways someone would actually try to cheat all fail against that verifier, and the prototype demonstrates each one.

TABLE 3 *Four ways to cheat — and how each one is caught.*

THE CHEAT	HOW IT'S CAUGHT
Edit an approved memo after the fact	The disclosed bytes no longer match the artifact's fingerprint.
Forge a record under a same-named lookalike CA	The certificate doesn't chain to the real CA, and the altered record isn't in the log.
Hide the failed attempt	The retry's "previous attempt" link points at a record that's missing.
Rebuild the whole log with one record swapped	The rebuilt history can't produce a consistency proof back to the notary's hash.

What building it taught us

I had the prototype adversarially reviewed, and it found two real bugs the original tests missed. I include them because the brief grades security awareness, and the honest form of security awareness is showing where you were wrong. Both were failures of attribution, the exact property the system exists to guarantee, and both were invisible to ordinary testing because they only surface when someone deliberately lies.

The first: expired certificates could pass on a broken timestamp. In JavaScript, every comparison against an invalid date returns false, so an unparseable timestamp read as being inside the validity window. Because a record carries a self-claimed time and the certificate is checked as of that time, a holder of an expired certificate could stamp a garbage time and slip through, which defeats the whole point of expiry-based rotation. The fix rejects unparseable times and fails closed.

The second: identity was signed but not bound. The verifier checked the signature against the named certificate and the certificate against the CA, but never checked that the identity claimed in the record matched the certificate's subject. So any valid key holder could attribute work to someone else, and an agent's own key could produce a human approval in a partner's name with every check passing. The fix requires an exact match on the claimed identity and delegation chain.

The lesson generalizes to any verification system, and it's the one I'd carry forward: the dangerous bugs are not in what the verifier checks, they're in what it silently forgets to check, and that gap only becomes visible when an adversary shows up. Which is why the most valuable tests in the suite are the ones that lie to it.

---

3 Tradeoffs

TABLE 4 *What the design optimizes for — and what it gives up, on purpose.*

OPTIMIZED FOR	GIVEN UP, ON PURPOSE
Verifiability by an outside party who doesn't trust Soma	Throughput (one write per second — nothing to optimize)
Provable completeness, via the Merkle log	Storage cleverness (small volumes; boring storage is easier to trust)
An insider with database access still can't rewrite history undetected	Sub-minute audit latency (indexes are rebuilt from the log, not kept always-correct)
Proofs that travel to the auditor, verifiable off Soma's systems	

Every one of those choices traces back to the single thing I optimized for: verifiability by a party who does not trust Soma. Content addressing so integrity is definitional, a Merkle log so completeness is provable, an external anchor so even an insider with database access can't rewrite history undetected, and a standalone verifier so the proof travels to the auditor instead of the auditor having to trust our systems. What I gave up, I gave up deliberately. Throughput was never in question at one write per second. Storage stayed boring on purpose, because small, boring storage is easier to trust than something clever. And I let audit queries take up to the minute the brief allows, which buys the freedom to rebuild query indexes from the log rather than maintain exotic always-correct ones.

The forks worth naming are the ones where a reasonable person might have gone the other way.

A **blockchain** instead of an anchored log would remove the trusted log operator entirely, which is a real property I give up. I passed because a permissionless consensus system and its availability dependency are wildly out of proportion for an internal tool at one write per second, whose rewrite window is already bounded to an hour by an outside notary. I kept it as an escape hatch: anchoring the top hashes into a public chain is one extra integration the day the trust requirements demand removing the operator.

A **lineage database plus a separate audit log** would be simpler to build, and I rejected it because it splits lineage and integrity into two systems that can drift apart, leaving the auditor to trust the join between them. Making the signed record stream itself be the lineage removes that seam.

Verifying AI steps by re-running them is the strongest possible check, and it stays available for deterministic steps. For LLM steps it can't work today, because re-running yields a different answer and providers don't sign their outputs, so the design commits to attributable-but-not-reproducible and records the full call context, with a clean upgrade to provider-signed inference the moment that exists.

The limitations are worth stating plainly, because an auditor distrusts a system that claims too much. It proves lineage, not truth: a hallucination is recorded with perfect integrity, which is why decision-grade artifacts require a human approval, putting an accountable person in every real decision chain. A live, compromised endpoint can sign real lies with a valid key; short certificate windows and the anchored log contain that but don't eliminate it. Outside data is trust-on-first-fetch, fingerprinted at the earliest moment so a dispute reduces to "this is exactly what their API returned at 14:02 UTC." And record timestamps are self-claimed, with the anchor proving only when a batch of history existed; per-record countersigned timestamps from the notary are the fix, and the interface is ready for them.

Finally, how it evolves, because the brief asks. The reassuring answer is that at 10x to 100x, nothing in the trust core changes shape; the record, the graph, the log, and the proofs stay exactly as they are. What grows is the plumbing around them. The log can be sharded per fund or per workflow class under a shared super-root so no single tree grows without bound, anchoring can be batched, cold artifact bytes can tier down to archival storage while their fingerprints stay hot, and the lineage view can move to a dedicated graph store, which is safe precisely because it's derived. The guarantee stays fixed while the machinery under it scales, which is the property you want from a trust system: what you could prove to the LP on the first day is what you can still prove on the ten-thousandth.